Logbook — Anomalous transport in soaring flights

Matteo Di Sante

Working log

Working notebook. The timeline is regenerated from the git history; the narrative notes are written by hand.

1 Timeline (auto-generated from git)

Date	Event (commit subject)
2026-07-06	fix(gap-fig): linear axis/bins for Fig 4.6 panel (a), reveals real small-integer structure
2026-07-06	fix(sampling-fig): discrete bins for Fig 4.7 (native dt), fraction not density
2026-07-06	fix(fixlevel-fig): shared 13 m/s vertical-speed bound; fix a general long-caption LaTeX limit
2026-07-06	fix(fixlevel-fig): integer-aligned bins for vz; revise horizontal-speed bounds to 45/55
2026-07-06	fix(fixlevel-fig): widen x-range for vxy/vz tails; explain vz's non-empty valleys
2026-07-06	feat(preproc): per-discipline fix-level speed bounds, on a much larger sample
2026-07-06	fix(altitude-noise): Fig 4.1 panel (a) back to dual y-axis, over the full 30-min window
2026-07-06	fix(thesis): Fig 4.1 redesign (drift over jitter), Appendix B panel-letter bug, missing sample sizes
2026-07-06	chore: add MIT LICENSE, README badges, and a real test-running CI
2026-07-06	ci(docs): auto-deploy on push again, scoped to paths that feed the site
2026-07-06	docs(logbook): reformat narrative notes for readability; re-sync Next steps; docs deploy on demand
2026-07-06	fix(altitude-noise): dual-axis + decoupled windows for Fig. 4.1 panels (a)/(b); chore(deps): dev/analysis as uv default groups
2026-07-06	docs: data-driven fix-level section; four-panel altnoise caption; blueprint, logbook, README
2026-07-06	chore(data): explicit SSD->repo seasons_index sync; document the offline snapshot
2026-07-06	feat(altitude-noise): four-panel figure (raw channels, difference, PSD, availability)
2026-07-06	feat(preproc): data-driven fix-level diagnostics; gap-histogram readability; drop redundant time-gap
2026-07-06	fix(igc): treat only full-day drops as a midnight roll-over
2026-07-06	chore: SSD data layout (raw/catalog/derived) + pyarrow for the scan cache
2026-07-05	thesis ch.4: data-driven pre-processing pipeline + geodesy figures
2026-07-02	Symmetric para/hang dataset; adopt barometric altitude; IGC parser + noise diagnostics

Date	Event (commit subject)
2026-07-01	Rename soaring-ffvl -> soaring-para; add hang-glider data and update all docs
2026-07-01	Fix delta config: season start 2001, data_root /Volumes/SSD_DISANTE/hang_gliders/delta_cfd_igc
2026-07-01	Add soaring-delta CLI for hang-glider CFD data (delta.ffvl.fr)
2026-07-01	Add analysis sub-package and preprocessing diagnostics; revise thesis chapters 1-4
2026-07-01	Expand pre-processing: explicit cleaning and filtering sections
2026-07-01	Clarify stationarity/compatibility check paragraph
2026-07-01	Add preliminary characterization section before main observables
2026-07-01	Add isotropy test protocol to the propagator item
2026-07-01	Rewrite propagator item: formal derivation of marginal scaling
2026-07-01	Reorganize TA-MSD item for clarity and typographic structure
2026-07-01	Clarify ergodicity breaking and add CTRW appendix
2026-06-28	Revise chapter 4 observables and the IGC description
2026-06-28	Warn when thesis and logbook next-steps drift apart
2026-06-28	Align logbook next steps with the thesis planning chapter
2026-06-28	Publish the logbook alongside the thesis
2026-06-28	Add living thesis document and build automation
2026-06-26	Deploy docs via GitHub Pages Actions artifact instead of gh-pages branch
2026-06-26	Fail fast with a clear message when data_root is unset
2026-06-26	Add 'verify' subcommand and remove machine-specific data_root path
2026-06-26	Link published docs in README; drop Development section
2026-06-25	Add GitHub Pages deployment via gh-pages branch
2026-06-25	Translate all documentation, comments, and strings to English
2026-06-25	Initialize thesis repo: FFVL CFD .igc data acquisition

2 Narrative notes

2026-06 — Paraglider data acquisition complete. The full FFVL paraglider CFD archive (`parapente.ffvl.fr`, seasons 1999–2000 to 2025–2026) has been scraped: about 203,000 flight records, of which ∼186,000 carry a GPS track. All 186,052 available tracks were downloaded.

What worked Impersonating a Chrome TLS fingerprint (`curl_cffi`) reliably passes the FFVL Cloudflare challenge; the resumable, atomic, validated download survived many interruptions without corruption.

Gotchas 173 tracks could not be obtained — 172 because the server returns something that is not a valid IGC (broken or placeholder links on FFVL's side, so a retry does not help), and 1 due to a transient Cloudflare challenge (worth a later retry). On macOS/exFAT the OS scatters `._*` sidecar files next to every written file; a `clean` step removes them.

2026-06 — Integrity verified. A full on-disk re-scan (`verify`) confirmed that all 186,052 downloaded files are structurally valid IGC: no truncated files, no leftover `.part`, no read errors. The verification logic was promoted from a throwaway script to a first-class `soaring-para verify` subcommand (reusing the download-time validator), with tests.

2026-06 — Robustness of the data path. The external disk remounted under a different name (`HDD_DISANTE` → `SSD_DISANTE`), which would have silently broken a hard-coded path. Fix: removed every machine-specific path from the repo (placeholder + `SOARING_PARA_DATA_ROOT`

environment variable) and added a start-up guard that aborts with a clear, actionable message when `data_root` is unset/unmounted — and verified that `/Volumes` is not user-writable, so a wrong path can never cause a silent download to the wrong place.

2026-07 — Hang-glider data acquired; naming corrected.

Second source added The hang-glider CFD (`delta.ffvl.fr`, seasons 2001–2002 to 2025–2026), sharing the acquisition code (`soaring.acquisition.ffvl`) via a second CLI entry point `soaring-delta` (config `delta_download.yaml`, env var `SOARING_DELTA_DATA_ROOT`). Data: ∼9,300 flights, 6,716 tracks downloaded, 30 failures (broken server-side links, same pattern as paragliders). Stored at `/Volumes/SSD_DISANTE/hang_gliders/delta_cfd_igc/`.

Naming corrected The paraglider CLI was renamed from the generic `soaring-ffvl` to `soaring-para` (and `SOARING_FFVL_DATA_ROOT` → `SOARING_PARA_DATA_ROOT`), and its config from `ffvl_download.yaml` to `para_download.yaml`, to avoid ambiguity now that both sources are present.

2026-07 — Hang-glider catalog built; datasets reorganized symmetrically. Ran the full pipeline on the hang-glider data (`soaring-delta clean`, `verify`, `build-catalog`, `status`), which so far had only been downloaded: 9,259 flights, 6,746 with a track, 6,716 downloaded and all verified (0 integrity failures), 30 unrecoverable server-side links.

Data layout The local `data/` folder now mirrors the SSD — one directory per discipline (`paragliders/`, `hang_gliders/`), each with a versioned `seasons_index.csv` and a git-ignored `catalog.csv`.

Thesis symmetry `generate_stats.py` now reads both indices and emits per-discipline macros (`\StatPara*`, `\StatHang*`) and two per-season tables; the first three thesis chapters were made symmetric across both disciplines.

2026-07 — Altitude channel decision: barometric. Decided to take the vertical dynamics from the *barometric* altitude rather than the GNSS altitude, diverging from the reference pipeline (Jérémie's code derives z and v_z from `AltGNSS`, reading but not using `AltPre`).

Rationale The pressure altitude has sub-metre relative precision and low high-frequency noise, its errors being slow (offset, drift) and killed by differencing, whereas the GNSS vertical carries a high-frequency noise floor that derivatives amplify. Confirmed empirically with a new noise diagnostic — an IGC `B`-record parser (`soaring.analysis.igc`) and an altitude-noise analysis (`soaring.analysis.altitude_noise`) whose Welch power spectra show the GNSS floor two to three orders of magnitude above the barometric one near Nyquist (thesis appendix on the method).

Consequences The `A/V` validity flag is subsumed by the missing-altitude check on the adopted channel (a `V` fix keeps position + baro on a baro flight, but is dropped on a GNSS-fallback flight); one altitude channel per trajectory, no baro/GNSS splicing; flights with no pressure sensor fall back to unfiltered GNSS, tagged by a per-flight `alt_source` column; the trimming threshold is on horizontal speed only ($v_{xy} > 5\,\mathrm{m/s}$, sustained); cleaning/trimming run on the raw geographic coordinates (great-circle speeds), with the ENU conversion done *last*, origin at take-off.

2026-07 — Baro-absence fraction: sized sampling, not a full sweep. A full-population parse of the paraglider archive (186,052 files) for the baro-channel-absence figure took an unacceptable ∼45 min serially; parallelising across 8 processes cut this to ∼15 min, still too slow to run routinely.

Fix A properly justified *simple random sample*: the standard proportion sample-size formula ($n = z^2 p(1-p)/e^2$, conservative $p = 0.5$) gives $n = 2401$ for a target 95%-confidence margin of error

of ±2 percentage points (`required_sample_size` in `soaring.analysis.altitude_noise`, with `proportion_ci` reporting the point estimate and interval). The population is treated as effectively infinite: at $n = 2401$ out of 186,052 flights, the finite-population correction would move n by only ∼30, not worth the extra parameter. The hang-glider archive (6716 files) stays a full census (cheap enough, under two minutes). Total runtime: ∼50 s. Result: paragliders (28.4 ± 1.8) % (95% CI, sampled), hang gliders 17.5 % (exact, census).

Also caught mid-flight The IGC parser accepted syntactically well-formed but semantically invalid `B` records (e.g. minute ≥ 60, hour ≥ 24, out-of-range latitude/longitude) without rejecting them; added explicit format-validity checks (9 new tests) and corrected the thesis wording, which had conflated this *implemented* format-validity layer with the fix-level *dynamics-plausibility* layer of Table 4.1 – at the time designed (a dataclass of thresholds) but not yet implemented as an enforced filter.

2026-07 — Pre-processing made data-driven and config-driven. Reworked Chapter 4 so the flight-level filtering is computed from the *full parsed dataset* (paragliders and hang gliders), not the declared catalog metadata.

What changed A track scan (`soaring.analysis.preprocessing.scan_tracks`) parses every downloaded track; Figure 4.2 shows the real duration and flown-path-length distributions with per-variable *marginal* retention curves. Adopted cuts: duration ≥ 40 min and path length ≥ 30 km (a minimal cut, ∼95% retained); the earlier min-fix-count cut was dropped as redundant with duration, and the *extent* proxy was replaced by total flown *path length* (median 86 km para, 158 km hang).

Key decision Every pre-processing threshold now lives in `configs/preprocessing.yaml` (never hard-coded), loaded via `load_preproc_config` into typed dataclasses.

Also added Great-circle (haversine) inter-fix speed/distance *before* the ENU conversion; a Savitzky–Golay smoothing/differentiation section with a noise-matched procedure for its two hyperparameters; the switch from a common resampling cadence to native per-flight Δt (Figure 4.5); and an implementation blueprint (`docs/guide/preprocessing-pipeline.md`) recording the exact schema and steps, to retire into code once the pipeline is built.

2026-07 — Gap diagnostics; scan cache; baro-presence consolidated. Added Figure 4.5, the sampling-regularity diagnostics (largest gap relative to native Δt, missing fraction of a uniform grid, each with its marginal retention curve), fulfilling a promise Sec. 4.1.6 already made but had not yet delivered.

Scan cache A fresh full parse would have repeated the ∼15–20 min paraglider census just done for Figure 4.2, so the per-flight scan is now **cached** to `data/<discipline>/track_scan.parquet` (gitignored, like `catalog.csv`; `load_or_scan_tracks` reads it if present, else scans and writes it — later moved to the SSD, see below). `track_stats` was extended with a few near-free byproducts of the same scan — barometric-presence fraction, max horizontal/vertical speed, barometric altitude range — for future validation of Table 4.1's fix-level bounds.

Consolidation The barometric-presence fraction (Sec. 4.1.1) now reads from this same cache (`altitude_noise.baro_presence_from_scan`) instead of running its own separate scan, turning the paraglider number from a sized sample into an exact census at no extra cost: 30.1 % (exact, $n = 186{,}025$) — consistent with the earlier sampled estimate, (28.4 ± 1.8) %.

Also caught A pre-existing fragility in `altitude_noise.collect` where a single unreadable file (a transient external-disk hiccup) aborted the whole PSD sample; it now skips and logs the file instead.

2026-07 — SSD reorganised; no data kept locally; figure fixes.

Policy going forward Raw data, catalogs and every analysis byproduct live only on the external SSD, never in the local repo.

Reorganisation Each discipline's `data_root` is now laid out by maturity — `raw/{igc,raw_xml}` (untouched acquisition output), `catalog/{catalog.csv,seasons_index.csv}` (tables derived from it), `derived/` (further byproducts, today just `track_scan.parquet`; future cleaned/filtered data and analysis results get their own `processed/`/`analysis/` siblings), with `Config` updated accordingly (new `derived_dir` property). Verified file counts before/after the move (paraglider: 186,052 tracks, 27 XMLs; hang glider: 6,716 tracks, 25 XMLs — all matched) and that both acquisition CLIs still work against the new layout. Previously-local `track_scan.parquet` copies moved to `<data_root>/derived/`; orphaned local `catalog.csv` copies (superseded by the track-based scan months ago) were removed.

Figure 4.5 (gaps) The adopted `max_gap_factor` was raised from 5 to 10 — at 5 it retained only 77.8 %/65.0 % of para/hang flights (nowhere near the near-full retention of the duration/path cuts), at 10 it reaches 86.9 %/85.1 %. Considered 20 (90.8 %/90.6 %, little further gain) and rejected it: the cut is *relative* to each flight's own native Δt, so for the slower hang-glider loggers (up to 10 s native) a factor of 20 would tolerate interpolating across a $\sim$200 s gap — long enough to span a thermal climb. Also tightened the distribution panels' axis range to the data's own percentiles (the old 0–200/0–0.6 range was mostly empty).

Figure 4.1 (altitude noise) Panel (a)'s difference trace now uses a colour distinct from both channels (was reusing GNSS's red); its window grew from 10 to 30 minutes; a Nyquist-frequency line was added to panel (b) (previously discussed in the caption but never drawn). Also found and fixed a real bug in the "representative flight" selection: picking the *longest flight by elapsed time* broke on a 30-hour, 1,100-fix outlier (one corrupted timestamp gap) — fixed to select by *fix count* instead, which has no such failure mode.

Local data folder clarified The two versioned `seasons_index.csv` are the offline reproducibility anchor for the thesis's dataset counts (the document builds with no SSD mounted), not stray local data. Their sync from the canonical SSD copy is now explicit and automatic (`refresh_seasons_index.py`, run best-effort by the pre-commit hook), so the snapshot cannot silently drift.

2026-07 — Fix-level cleaning made data-driven; parser roll-over bug; figure and data-folder cleanups.

Fix-level cleaning audited Table 4.1 is now audited on the data, like the flight-level cuts: a new figure in Sec. 4.1.2 shows the per-*fix* distributions of horizontal speed, barometric vertical speed and barometric altitude (`make_fixlevel_diagnostics_figure`, from `fix_level_distributions` over a seeded sample — millions of fixes). What justifies a fix-level bound is the fraction of *fixes* it removes, not the number of flights it touches; the per-flight *maxima* already in the scan (`max_vxy_mps` and friends) answer the wrong question, so they do not place the cuts.

Gap handling unified Removed the absolute 60 s inter-fix time-gap from the fix-level table: it duplicated the sampling-regularity cut of Sec. 4.1.6, which is more principled (relative to each flight's own native Δt). Neither was ever an enforced filter, so this is a spec simplification, not a behaviour change.

Parser bug fixed Found the root cause of the "30-hour, 1,100-fix" pathology worked around previously: the IGC parser's midnight roll-over unwrap treated *any* backward step in time-of-day as a new day and added 86,400 s, so a few-second GPS glitch became a spurious multi-hour gap. Now only a drop of (nearly) a whole day counts as a roll-over, with residual backward jitter clamped to keep the series monotonic. Added regression tests for both the true roll-over and the glitch.

Figure and config cleanups The altitude-noise figure (Sec. 4.1.1) is now four panels — (a)

raw channels, (b) their difference, (c) PSD, (d) fallback rate — fixing the old a/b/d labelling. The gap-diagnostics figure (Sec. 4.1.6) no longer clips its tail into an artificial edge spike, with an x-range fitted to the data's own percentiles. Result-specific numbers (retention percentages, one-off anecdotes) were pruned from YAML/code comments — such numbers belong in the thesis, not in reusable config.

Local data folder clarified Rewrote `data/README.md` and fixed several stale references to a flat `data/seasons_index.csv` path that never existed.

2026-07 — Coordinate/geodesy figures. Redrew the coordinate-transformation figures: Figure 4.3 (WGS84 ellipsoid; geodetic vs geocentric latitude) and Figure 4.4 (the ECEF→ENU rotation, in our notation), and added the ESA Navipedia citation for the ECEF conversion.

2026-07 — Altitude-noise figure (Fig. 4.1): panels (a)/(b) redesigned. A closer look at the rendered figure (not caught by any test, since nothing there errors) found two compounding problems in the one-flight time-series panels, both traced to a single shared 30-minute window: the several-hundred-metre climb dominated the vertical scale, hiding the GNSS jitter the panel exists to show; and panel (b), subtracting only the scalar *mean* of the GNSS−barometric difference, ended up on a ±20 m scale that looked inconsistent with the ∼130 m gap plainly visible in panel (a).

First attempt (reverted) Shortened the shared window to 2 minutes and switched panel (b) to a *linear* detrend (matching `_welch`'s). This resolved both symptoms but destroyed something worth keeping: the GNSS−barometric difference actually drifts by tens of metres over a full flight (plausibly the barometric channel's departure from the ICAO standard atmosphere as altitude changes) — a real, physically meaningful feature that a 2-minute window is too short to show and a linear detrend actively removes.

Final design Decoupled the two panels' windows and mechanisms instead of forcing one compromise on both. Panel (a) keeps a short (2-minute) window but now plots barometric and GNSS on *two independent y-axes* (same numeric span, different baseline) rather than one shared axis, which previously had to stretch from one channel's minimum to the other's maximum — wasting on blank space exactly the room needed to compare the two channels' roughness; with matched spans the GNSS trace is visibly more jagged than the barometric one. Panel (b) reverts to the original 30-minute window with only the *mean* removed (no detrend), so the slow drift is visible again alongside the residual jitter, with the removed mean offset (139 m for this flight) annotated directly on the panel. Both panels' wording was generalised from "climb" to "vertical motion", since a short window can just as well land on a glide.

2026-07 — `dev`/`analysis` moved from pip extras to uv default dependency groups. Chasing the figure fix above, a bare `uv sync -reinstall-package soaring` (no `-extra` flags) silently reset the environment to the core-only dependency set, uninstalling `pytest`/`ruff`/`mypy`/`matplotlib`/`scipy`/`pyarrow` — and, on the way, hit a pre-existing `uv` bug (uninstall failing on packages whose `RECORD` file was missing) that required rebuilding `.venv` from scratch.

Root cause `dev` and `analysis` were plain PEP 621 `optional-dependencies`, so *any* `uv sync`/`uv run` invocation without matching `-extra` flags resyncs the venv down to exactly what was requested, silently dropping whichever extras the previous invocation had added — alternating flagged and unflagged commands across a session means constant install/uninstall churn.

Fix Moved both to `[dependency-groups]` (PEP 735) with `[tool.uv] default-groups = ["dev", "analysis"]`, so every `uv sync`/`uv run` includes them unconditionally. Neither group was ever meant to be pip-installed by an external consumer in practice (this repo is

not published), so the loss of `pip install .[dev]`-style installability is immaterial; `docs` remains a real extra since it is genuinely on-demand. The docs-deploy CI workflow now opts out of the new default groups (`-no-default-groups`) to stay minimal.

Docs updated `README.md`, `docs/guide/installation.md` (dependency-groups table, pip fallback instructions), the two reporting scripts' now-unnecessary `uv run -with matplotlib -with scipy` incantations, and the `soaring.analysis` package docstring.

2026-07 — Logbook readability; Next-steps drift; docs deploy on demand. Three unrelated loose ends, tidied together.

Logbook typography The narrative notes below were dense, single-paragraph entries — hard to scan. Reformatted the longer ones into short lead sentences plus labelled sub-points (*what changed*, *why*, *gotchas*), and dropped a stale forward-reference in the altitude-channel entry ("to confirm on a larger sample once the SSD is remounted", superseded two entries later by an exact 186,025-flight census).

Next-steps drift The Next steps list below had fallen out of step with the thesis chapter it is meant to mirror: *Preliminary characterization* (thesis Sec. 4.2) and *Tooling* (Sec. 4.9) were missing entirely, and the whole-trajectory, phase-resolved and transport-analysis bullets were missing content the thesis already has (the propagator/return-probability/first-passage observables; the step–waiting coupling and phase-sequence memory; the stratification and bias-control plan). Re-synced against the current chapter.

Docs deploy The GitHub Pages workflow redeployed on every push to `main`, including pushes with nothing to do with the site (e.g. acquisition/analysis code with no docstring change) — not wanted. One such deploy had also failed with a generic, transient "try again later" from GitHub's Pages API, but that was an isolated glitch, not a config problem (confirmed via the Actions log: only that one run failed out of the last ten, and only at the deploy step, after a successful build). Fix: the `push` trigger is now scoped with `paths` to what actually feeds the built site (`docs/**`, `mkdocs.yml`, `src/soaring/**`, whose docstrings mkdocstrings renders into the API reference), so the site still updates itself automatically but only for changes that matter to it; `workflow_dispatch` stays available for an on-demand redeploy.

2026-07 — License added; README badges; a real test-running CI. `pyproject.toml` had declared `license = MIT` since the project's start, but no `LICENSE` file existed, and the repo had no workflow that actually ran the test suite (only the docs-deploy one).

License Added `LICENSE` (MIT): permissive, minimal text, the de facto standard for research Python code, and consistent with what `pyproject.toml` already declared – no reason to pick something more restrictive for code meant to be freely reused.

New CI: `tests.yml` Runs `uv run pytest` with coverage on every push to `main` and every PR. Needed no `-extra` flags to get `pytest-cov/matplotlib/scipy/pyarrow` – a direct payoff of making `dev/analysis` uv default groups earlier. Coverage XML uploads to Codecov as a best-effort step (`fail_ci_if_error: false`): a Codecov hiccup, or the repo not being linked there yet, must never redden the Tests badge.

README badges Added five badges (python version, Tests workflow status, Codecov coverage, docs-online link, license) at the top of `README.md`. The Codecov one needs a one-time manual step outside this repo – linking `matteodisante/soaring-anomalous-transport` on `codecov.io` (GitHub login) and, if it asks for one, adding `CODECOV_TOKEN` as a repo secret – which cannot be done from here.

2026-07 — Fig. 4.1/4.2 and Appendix B: closer reading, several fixes. A line-by-line pass over the altitude-noise figure, the fix-level figure, and the PSD appendix, prompted by direct questions about what they actually show.

Fig. 4.1 panel (a), third look The dual-axis, short-window version (see the entry above) did not read, on the page, as clearly showing GNSS is the noisier channel, so it was dropped for a shared axis over the full 30 min window instead – which showed the offset drift directly, but on request the panel went back to a dual axis once more, this time over that same 30 min window rather than a short one. Net result: each channel is independently auto-scaled (barometric left, GNSS right), so the two curves track almost on top of each other (both are the same ∼500 m climb, just at a different baseline) – good for reading each channel's own shape, but it means the raw vertical gap is no longer a literal reading of the offset. That number was never really this panel's to carry: it lives in panel (b), and the noise-floor claim in panel (c), regardless of which of these three versions panel (a) went through.

Appendix B (PSD) bug Two cross-references pointed at "Figure 4.1(b)" for the PSD, which has been panel (c) since the four-panel redesign; fixed. Also added the actual flight yield of the Δt-matching step, which the text described only qualitatively: of 400 sampled flights per discipline, 212 paragliders but only 114 hang gliders pass it, because hang gliders split across several native rates while paragliders are overwhelmingly 1-second loggers – so the hang-glider PSD curve is honestly an average over that 1-second minority, not the full hang-glider population, now stated as such.

Fig. 4.2 (fix-level), added The exact sample (800 flights per discipline, 6,633,923/4,541,511 paraglider/hang-glider fix pairs) was missing from the caption; added. Also explained two things a reader flagged as confusing on sight: panel (a) (horizontal speed) shows a knee well below the adopted 40 m/s bound (∼23/35 m/s for para/hang) – read as the transition from ordinary gliding speeds to rarer but real high-energy manoeuvres (wind-assisted glides, steep spirals) rather than GPS error, hence the bound is deliberately kept above it, not moved down to it, so as not to discard genuine dynamics; and panel (b) (vertical speed)'s comb-like histogram is a quantisation artefact (integer-metre barometric altitude differenced over a 1-second native step can only return an integer m/s), not a plotting bug.

Table 4.1, added A one-sentence justification for the −100 m lower altitude bound, which read as arbitrary: it is not a claim about true terrain elevation, but headroom for the barometric channel's own non-QNH-corrected offset (Figure 4.1 measures this directly, ∼100 m for the flight shown there), so a genuine sea-level take-off can log slightly negative without being cut.

2026-07 — Fix-level speed bounds made per-discipline, on a much larger sample. Chasing the loosened-threshold experiment of the previous entry to a conclusion.

Sample size `FIXLEVEL_SAMPLE_PER_DISCIPLINE` raised from 800 to 15,000: 15,000 paragliders (126,723,499 fix pairs, ∼73 s) and all 6,716 hang gliders (a full census at this population size, 37,915,334 pairs, ∼25 s) – under two minutes total, and it resolves the sparse tail well enough to distinguish real dynamics from a rare artefact, which the 800-flight sample could not.

The hang-glider bump, confirmed and explained At this sample size the 45–90 m/s feature is unambiguous: not a smoothly decaying tail but a distinct, much more slowly-decaying plateau (confirmed with a fine, 2.5 m/s-binned histogram – the decay rate visibly changes between 37.5 and 42.5 m/s, from a steep real decline to a near-flat plateau lasting to ∼90 m/s). A sustained *ground* speed of 160–324 km/h has no credible airborne explanation for a hang glider: read as a logger/GPS artefact population, not rare-but-genuine manoeuvres.

Decision: per-discipline bounds, not one shared value Paragliders and hang gliders have different performance envelopes, confirmed by their different tail shapes – a shared bound is either too loose for one or clips real dynamics of the other. `FixLevelThresholds.max_horizontal_speed_mps`/`max_vertical_speed_mps` are now `dict[str, float]` keyed by discipline (`configs/preprocessing.yaml`, a nested mapping; a `"sailplanes"` entry can be added later with no code change). Vertical speed: 12 m/s (para) / 14 m/s (hang).

Horizontal speed bound, revised again: 40 was still "slowing", not "stopped" First adopted

at 25/40 m/s (para/hang), where the fine-binned decay rate first visibly changes. On review, with the panel's x-range widened (next entry) to actually show the far tail, that point turned out to still be on the shoulder of the decline, not the flat artefact itself: the bin-to-bin ratio only settles at ≈ 1 (a genuine plateau, decay actually stopped) around 55 m/s for hang gliders, and the analogous point for paragliders is ≈ 45, not 25. The 25/40 m/s pair was therefore cutting into the tail-end of real (if rare) dynamics – a wind-assisted glide, a steep spiral – rather than sitting cleanly past it. Revised to **45 m/s (para) / 55 m/s (hang)**, both now Table 4.1's and the config's adopted values.

Figure and table updated `make_fixlevel_diagnostics_figure` now draws one dashed cut per discipline, colour-matched to its histogram, with a per-discipline removed-fraction annotation (previously one shared cut and fraction). Table 4.1 gained a row per discipline for the two speed bounds.

−100 m altitude bound, empirical check (unchanged, kept for the record) Computed the per-flight mean GNSS−barometric offset over 1500 flights per discipline (broader than the single Figure 4.1 example): median ≈ 90 m, 25th–75th percentile roughly 43–135 m. This is a *whole-flight* average, and the offset itself grows with altitude (Sec. 4.1.1), so it likely *overstates* the offset specifically near the ground; confirms the order of magnitude of −100 m without pinning it down precisely. No error found in Figure 4.2 itself.

x-range widened for (a)/(b) The panels' range was cropping the view almost exactly at the cut (the 99.9th percentile of the pooled sample sits right there, by construction), hiding how much – and what – the bound leaves out. Panels (a)/(b) now extend to each discipline's own 99.99th percentile (computed per discipline, not pooled, so hang gliders' heavier tail is not swamped by paragliders' larger sample); panel (c) (altitude) keeps the original, narrower range on purpose – its cuts sit far out in an otherwise tight, unimodal distribution, and widening it the same way mostly shrinks the informative part of the panel.

Panel (b)'s non-empty valleys, explained and checked The comb (spikes at integer m/s) was expected; that the valleys *between* integers were not exactly zero was not, and looked like a bug. It is not: not every flight samples at 1 s (Figure 4.7 – hang gliders spread across several native rates), and differencing over a longer native step returns a finer-grained set of values (e.g. multiples of 0.2 m/s at a 5 s cadence) that fill the gaps once every cadence is pooled. Checked directly, split by native rate: among 1 s-native fixes, ∼99% land within 0.05 m/s of an integer (the comb, essentially pure); among every other native cadence, only 34–42% do – the other 58–66% is exactly what shows up between the teeth.

Panel (b)'s jagged comb, fixed Separately from explaining *why* the valleys were not empty: the comb itself looked bad – an arbitrary, fine (linspace) grid of bins straddles the integer spikes inconsistently depending on where each bin edge happens to fall, which reads as noisy up-down jitter rather than the smooth decaying envelope the data actually has. Fix: one bin per integer, centred on it (edges at half-integers) rather than an arbitrary edge grid – `make_fixlevel_diagnostics_figure` now takes an `integer_aligned` flag, set for panel (b) only (vxy is not similarly quantised). The panel is now a clean, monotonically-decaying curve; the removed-fraction numbers are unaffected (computed from the raw data, not the histogram).

Vertical speed reverted to one shared bound: 13 m/s Unlike horizontal speed, paragliders' and hang gliders' $|v_z|$ distributions sit close enough together (panel (b)) that a per-discipline split buys nothing; a single 13 m/s bound (between the previous 12/14 split) covers both, with one shared grey dashed line and one pooled removed-fraction annotation, mirroring how the altitude band already works. `FixLevelThresholds.max_vertical_speed_mps` reverted from `dict[str, float]` to a plain `float`; the figure's two shared-cut helpers (band and single-cut) were merged into one `_shared_cut_panel` that handles either, so panel (b) still gets the `integer_aligned` treatment above.

A LaTeX limit hit, and fixed for good: very long captions Rebuilding after the above changes hit `! Dimension too large` / "I can't work with sizes bigger than about 19 feet", deep in `\@makecaption`. Bisected the Figure 4.2 caption by half repeatedly to isolate it – not a typo (braces and math-mode $ were balanced throughout), but the caption's sheer length (∼3900 characters): plain LaTeX's single-line-fit check for captions measures the *whole, unwrapped* caption as one hbox before deciding whether to wrap it, and past a few thousand characters that measurement alone overflows TeX's ∼19-foot maximum dimension, even though the wrapped result would render fine. Loading the `caption` package does the same check by default (to centre genuinely short captions), so `singlelinecheck=false` was needed on top of it. This is a general limit of this document now that several figures carry long, data-justified captions – not specific to this one – so the fix (`main.tex` preamble) covers all of them going forward, not just Figure 4.2.

2026-07 — Figure 4.7 (native sampling interval): discrete data, discrete plot. Raised independently: the density in Figure 4.7 rose above 1, and native Δt is a discrete quantity, so why plot a density at all. Checked on the full census: Δt lands on an exact whole second for 99.9% of paragliders and 100% of hang gliders – essentially a categorical variable (one of a handful of configured logger rates), not a continuous one. The previous plot used arbitrary 0.5 s-wide bins on this discrete data (`np.arange(0.25, 8.75, 0.5)`): most paragliders' mass concentrated in the single bin straddling $\Delta t = 1$, and `density=True` divides a large fraction by that narrow bin width, which is exactly how it exceeded 1 – a real number, not a bug, but a genuinely confusing one for data this concentrated. Fixed the same way as the vertical-speed comb: bins one full second wide, centred on each integer ($1, \ldots, 10$, plus a ≥ 11 bin for the long thin tail), so each bar reads directly as "fraction of flights at exactly this rate." With unit-width bins `density=True` is numerically identical to that fraction (dividing by a width of 1 changes nothing), so the axis is now labelled *fraction of flights*, not *density* – accurate either way, but only one of the two labels says what is actually being counted. Real distribution, now stated precisely in the caption instead of the vaguer "$1, 2, 5, \geq 8$" before: paragliders overwhelmingly (69%) at 1 s; hang gliders spread across 1 s (30%), 5 s (27%), 10 s (15%), 2 s (13%), and a long thin tail beyond 11 s – a fuller picture than the old caption's "up to 8 s," corrected in Sec. 4.1.6 too.

2026-07 — Figure 4.6 (gap diagnostics), panel (a): dropped the log axis. Same family of question as the two entries above, this time about the largest-gap-to- native-Δt ratio: is it also discrete, why a density, and (specifically here) is a log x-axis actually needed.

Not as clean-cut as native Δt Checked directly: only 86% of paraglider and 63% of hang-glider values are exact integers (thousands of distinct values in all, tail out to $>50{,}000$) – a missed fix or two at the native rate is the single most common cause of a large gap (hence the pull towards small integers), but plenty of gaps are not a clean multiple of it. So, unlike Figure 4.7, this is not purely categorical; integer-only bins would have thrown away a genuine continuous component.

The log axis, though, was not earning its keep Panel (a) only ever displays the *bulk* of the distribution (up to `gap_hi`, a modest ∼1–20 here) – the heavy tail is deliberately left to the retention panels (c)/(d), which already sweep the full range on a log axis, justified there since the cut spans orders of magnitude. Over the narrow, already-cropped range panel (a) actually shows, a log axis buys nothing and actively hides structure: log-spaced bins are extremely narrow near 1, so the large mass sitting at small integers – a real, common failure mode, not noise – gets compressed into a sliver.

Fix Panel (a) switched to a linear axis with linear, evenly-spaced bins over the same `gap_hi`-bounded range. This did not just fix the axis choice: it revealed a real pattern that the log/narrow-bin combination had been washing out – visible peaks at small-integer ratios, most clearly for hang gliders (around $k = 3, 6, 10, 15$), consistent with the "missed native-rate

fixes" mechanism. Panel (c)'s log axis is unaffected and still the right choice there, for the opposite reason: it has to show a cut ranging over orders of magnitude.

2026-07 — Why gap ratios aren't integers: stratified by native rate. Follow-up on the panel (a) entry above: if a missed native-rate fix is the single most common cause of a large gap, why is the largest-gap ratio an exact integer only 86%/63% of the time, not effectively always? Stratifying the full census by native Δt answers it.

$\Delta t = 1$ s is trivially always integer Every IGC timestamp already has whole-second resolution, so the largest gap itself is always a whole number of seconds; dividing it by $\Delta t = 1$ cannot produce anything but an integer. Checked directly: 100% for both disciplines at this rate. Since 1 s is also the dominant paraglider rate (69% of flights, Figure 4.7), it alone pulls the paraglider-wide fraction up to 86%.

Slower configured rates are only about half integer At $\Delta t = 2/5/10$ s the integer-ratio fraction is 60%/58%/51% for paragliders and 34%/61%/39% for hang gliders – no rate stands out, none close to 100%. The mechanism from the panel (a) entry still holds (a missed fix is still the main driver of a large gap), but the gap itself is a real elapsed duration – GPS reacquisition time, logger buffering, a signal dropout – set by whatever caused the interruption, not by the logger's configured cadence. Nothing requires that duration to land on a multiple of Δt; it does so about half the time by chance, not by construction.

So the headline 86%/63% is a mixture effect Paragliders are predominantly 1 s loggers (trivially integer), hang gliders are spread across several slower rates (mostly not) – the two overall percentages mainly reflect which rate dominates each fleet, not some fleet-wide regularity in how gaps occur.

2026-07 — Figure 4.6, panel (b): tightened the missing-fraction axis. The x-axis ran out to roughly twice the adopted cut (0.10), leaving the right half of the panel empty: the 90th percentile of the pooled missing fraction sits at ≈ 0.09995, essentially at the cut itself, so nothing of substance lives beyond it. Changed the margin past the cut from 2× to 1.1×, i.e. `miss_hi = max(p90, cut * 1.1)`, giving a 0–0.11 range that still clears the 90th percentile with a little headroom but no longer wastes half the panel on empty axis.

2026-07 — Figures now stay where they are placed in the source. Plain LaTeX's float algorithm was free to defer any figure past its ideal spot to the next page it judged to have room, and with several large figures placed close together in Sec. 4.1 it was pushing some many pages away from the text discussing them – annoying for anyone reading top to bottom rather than jumping via cross-references. Added the `float` package and switched every `figure`/`table` placement specifier (previously `[t]`/`[!]`) to `[H]` ("here, literally"), which pins each one to its position in the source, full stop. No layout regressions on a full rebuild: text and figure land on the same page throughout, including the two cases checked directly (Figures 4.6 and 4.7, immediately after the paragraphs that introduce them).

2026-06 — Documentation. Project docs are built with MkDocs and published on GitHub Pages. Initially deployed with `mkdocs gh-deploy`, which accumulated one built-site commit per deploy on a `gh-pages` branch; migrated to the official GitHub Pages *artifact* workflow (build → upload artifact → deploy), removing the branch and keeping the git history clean.

Operational note. In this repo `uv sync` occasionally fails to (re)install the editable package; `uv sync -reinstall-package soaring` fixes it.

3 Next steps

These mirror the planning chapter of the state document (`thesis/sections/04-next-steps`, itself titled "Next steps") and are kept in sync with it, section by section. The near-term plan is to analyse the data *in parallel* with studying the segmentation, since many observables can be built before any phase splitting.

- **Data sources.** FFVL paraglider and hang-glider CFD data are collected, cataloged and verified. Extend to further sources – notably WeGlide for sailplanes (a higher-quota API request is pending; the default rate limit makes a bulk export infeasible) and possibly XContest – merging into one catalog and stratifying by source. Refresh the per-discipline `seasons_index.csv` via `build-catalog` so the document statistics stay current.

- **Pre-processing (thesis Sec. 4.1).** The parser is in place (`soaring.analysis.igc`) and the pipeline is designed, thresholded (`configs/preprocessing.yaml`) and audited on real data: fix-level cleaning (great-circle speeds), v_{xy}-based trimming, data-driven flight-level filtering (duration + path length), the ENU conversion (barometric vertical), native-Δt uniformity, and Savitzky–Golay smoothing/differentiation. Next: implement it end-to-end as production code (parser $\to \ldots \to$ Parquet `fixes/flights_meta` tables) per the blueprint, and measure the real ENU power spectra to finalize the two Savitzky–Golay smoothing timescales (still placeholders in the config).

- **Preliminary characterization (Sec. 4.2).** Lightweight, segmentation-free checks before the main analysis: per-flight trajectory statistics (duration, path length, fix count, native Δt); an isotropy check (the per-component variance ratio $\langle E(t)^2\rangle/\langle N(t)^2\rangle$, expected identically 1, plus a Kolmogorov–Smirnov test) needed before the marginal-propagator scaling can be used; compatibility across strata (glider class, season) via overlaid per-component displacement-variance curves, to justify – or rule out – pooling them; and a spatial-coverage check (take-off and displacement maps) for possible geographic stratification.

- **Whole-trajectory observables (Sec. 4.3, no segmentation).** Characterise the data with observables that need no phases: the MSD and its scaling exponent α (the primary anomalous-transport test); the time-averaged MSD, whose agreement or disagreement with the ensemble MSD is an ergodicity test; the one-dimensional marginal propagator and its scaling collapse; the return probability and the non-Gaussian parameter; the velocity autocorrelation and turning-angle distribution; first-passage/first-exit times; the speed/vertical-velocity distributions; and geometric measures (tortuosity, fractal dimension, radius of gyration), read both as one number per flight and as functions of scale. Goal: a first physical/statistical picture, independent of any phase splitting.

- **Segmentation (Sec. 4.4).** Study Jérémie's internship report and code, then re-examine the transition/search/climb HMM (binary features: sign of vertical speed, curvature-angle persistence, straightness) — decide whether to keep that approach or add complexity for a better segmentation on our larger, more heterogeneous data.

- **Phase-resolved observables (Sec. 4.5).** After segmentation: the waiting-time decomposition (climb τ_c, search τ_s, and their sum τ_w), the glide step-length/duration joint law, per-flight anatomy (thermal count, phase time/distance budget), and the angular diffusion between successive glides. Plus two couplings that decide which transport model applies: the step–waiting coupling (ℓ vs τ_w, the Lévy-walk signature) and memory in the phase sequence (transition-matrix structure, duration correlations), which test the renewal assumption behind the textbook CTRW/Lévy-walk results.

- **Reproduce the reference (Sec. 4.6).** Check that the enlarged paraglider-and-hang-glider dataset reproduces Vilpellet et al.: per-phase conditional MSD (their Fig. 3) and

per-phase descriptive statistics (their Fig. 4), plus the global $H \approx 0.88$ — validation and universality test (the sailplane comparison becomes reachable once that source is in hand).

- **Transport analysis (Sec. 4.7).** Estimate the MSD scaling exponent, fit the tails of the waiting-time and step-length distributions with principled methods (e.g. maximum-likelihood power-law fits with goodness-of-fit tests), and perform CTRW-vs-Lévy-walk model selection using the step–waiting coupling. Results will be stratified (season, site, glider class, data source) and trajectories normalized, with finite-size, censoring and sampling biases controlled throughout.
- **Minimal model (Sec. 4.8).** A minimal stochastic model for $H \approx 0.88$: resolve transition segments, merge search+climb into one effective state, with angular diffusion between successive transition directions (preliminary idea, to be refined/changed).
- **Tooling (Sec. 4.9).** Largely underway already: the analysis package mirrors acquisition, and the dataset statistics plus several diagnostic figures (pre-processing, altitude noise) already regenerate automatically from the data rather than being pasted in by hand. Remaining: extend the same treatment to whatever tables/figures the transport analysis produces, once it exists.